

Pandas Cheat Sheet

1. First Look at the Data

Examine first 5 rows

```
df.head()
```

Shape of the DataFrame

```
df.shape
```

This gives number of rows and columns as a tuple.
Example: (100, 5) means 100 rows and 5 columns.

Data types of the columns

```
df.dtypes
```

Summary statistics for numeric columns

```
df.describe()
```

2. Selecting Columns

One column

```
df["Score"]
```

This returns one column. The result is a Series.

Multiple columns

```
df[["Score", "Category"]]
```

This returns a DataFrame with just those columns.

Important

- One column = one set of brackets
- Multiple columns = two sets of brackets

3. Selecting Rows with iloc

Use iloc when you want to select by position number.

First row

```
df.iloc[0]
```

Rows 0 through 4

```
df.iloc[0:5]
```

First column

```
df.iloc[:, 0]
```

Rows and columns by position

```
df.iloc[0:5, 0:2]
```

(rows 0 through 4 and columns 0 and 1)

4. Boolean Masks (Filtering Rows)

A boolean mask is a column of True/False values.

One condition

```
df["Score"] > 90 #just the mask
df[df["Score"] > 90]
#use the boolean mask to extract values too!
```

Two conditions with AND

```
df[(df["Score"] > 90) & (df["Country"] == "USA")]
```

Two conditions with OR

```
df[(df["Category"] == "Solo") | (df["Category"] == "Duo")]
```

Rule: Use & for AND, Use | for OR, Put each condition in parentheses

5. Selecting Rows and Columns Together

Use a mask and show only certain columns

```
mask = df["Score"] > 90
df.loc[mask, ["Dancer", "Score"]]
```

This filters rows AND keeps only the selected columns

6. Creating / Adding Columns

Create a new numeric column

```
df["Score_Double"] = df["Score"] * 2
```

Create a boolean column

```
df["Top_3"] = df["Ranking"] <= 3
```

More Advanced: if/else setting values

```
df["Result"] = "Other" #set all to the same
df.loc[df["Ranking"] <= 3, "Result"] = "Top 3"
#if Ranking is less than 3, make text "Top 3" in Result column
```

7. Working with Text (Strings)

Remove spaces

```
df["Country"] = df["Country"].str.strip()
```

This removes spaces at the beginning and end.

Make lowercase

```
df["Country"] = df["Country"].str.lower()
```

Make title case

```
df["Country"] = df["Country"].str.title()
```

Replace text

```
df["Country"] = df["Country"].replace("USA", "United States")
```

IMPORTANT: Reassign the column

```
df["Country"] = df["Country"].str.strip()
```

Below will NOT permanently change the column:

```
df["Country"].str.strip() #no permanent changes
```

8. Changing DataTypes

Convert text to numeric

```
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
```

errors="coerce" means convert what you can AND bad values become NaN

9. Counting Values

Count the number of values per Category

```
df["Category"].value_counts() #result is a series
```

10. Sorting

Sort by one column

```
df.sort_values("Score") #sort small to large
```

Sort descending

```
df.sort_values("Score", ascending=False)
```

Sort by more than one column

```
df.sort_values(["Category", "Ranking"])
```

This sorts by Category first, then Ranking inside each Category.

11. Basic Summary Functions

Average

```
df["Score"].mean()
```

Smallest value

```
df["Score"].min()
```

Largest value

```
df["Score"].max()
```

These are useful for quick summaries of one column.

12. GroupBy

Use `groupby()` when you want a result for each category.

Average score for each category

```
df.groupby("Category")["Score"].mean()
```

Min and max for each category

```
df.groupby("Category")["Score"].agg( ["mean", "min", "max"])
#Nice to know if you want to do more than one thing!
```

Group by two columns

```
df.groupby(["Category", "Country"])["Score"].mean()
```

This gives the average score for each Category-Country combination.

13. reset_index()

After `groupby()`, the grouping column often becomes the index.

```
result = df.groupby("Category")["Score"].mean()
result = result.reset_index()
```

Use `reset_index()` to turn the result back into a normal table.

14. idxmin() and idxmax()

These return the row label where the smallest or largest value occurs.

Row label of smallest score

```
df["Score"].idxmin()
```

Row label of largest score

```
df["Score"].idxmax()
```

Use it to get the full row

```
i = df["Score"].idxmax()
df.loc[i]
```

This gives the whole row for the maximum score.

15. Best Row in Each Group

This is very useful when you want one best row per category.

Use `idxmin()` for best ranking

```
idx = df.groupby("Category")
["Ranking"].idxmin()
df.loc[idx]
```

This finds the row with the smallest ranking in each category.

Sort then take the first row

```
sorted_df = df.sort_values( ["Category", "Ranking"])
best_rows = sorted_df.groupby("Category").first()
```

This works because sorting puts the best row first in each group.

16. Common Patterns to Remember

Filter rows

```
df[df["Score"] > 90]
```

Average by group

```
df.groupby("Category")["Score"].mean()
```

Find the row with the biggest value

```
i = df["Score"].idxmax()
df.loc[i]
```

Find the best row in each group

```
i = df.groupby("Category")["Ranking"].idxmin()
df.loc[i]
```